Predicción de popularidad de canciones - Sprint 2

Integrantes:
Xavier Alexander Bravo Berroterán
Carlos Alberto Mancilla Valle
Esteban Vallejo Galeano
Materia: Data Analytics

1. Introducción

El presente informe documenta el Sprint 2 del miniproyecto «Predicción de popularidad de canciones», desarrollado en la materia Data Analytics. En el Sprint 1 se estimó la popularidad de una canción en una escala de 0 a 100. En este segundo sprint el enfoque se transforma en un sistema de decisión de negocio: antes del lanzamiento, el modelo determina si una canción tiene el potencial de convertirse en un éxito comercial (es_hit = 1 cuando la popularidad alcanza o supera los 70 puntos) y, con base en ello, recomienda si la disquera debe invertir en su promoción.

Como los errores de clasificación no cuestan lo mismo, se definió una función de costo asimétrica: ignorar un éxito potencial (falso negativo) cuesta tres veces más que promocionar una canción que no despega (falso positivo). Por ello el modelo no busca maximizar la exactitud, sino minimizar el costo total sobre la partición de prueba temporal (2020-2025). Se entrenó un HistGradientBoostingClassifier con validación temporal, se determinó un umbral de decisión de 0,15, se expuso el modelo mediante un servicio FastAPI desplegado en Render y se complementó con un panel interactivo en Streamlit.

El contenido se elabora a partir de los artefactos del proyecto: la ficha del modelo generada por src.entrenar_clasificacion, el cuaderno de entrenamiento, los archivos requirements.txt y render.yaml, el código del panel (app.py), el README y el conjunto de datos canciones.csv. Las Figuras 2, 3 y 4 se calcularon directamente sobre ese conjunto de datos; las Figuras 5, 6 y 7 representan las cifras reportadas en la ficha del modelo.

Tabla 1. Resultados clave del Sprint 2 en el periodo de prueba (2020-2025)

Indicador	Resultado	Interpretación
Costo financiero total	318 → 107 unidades	Reducción del 66,3 % frente a la línea base
Sensibilidad (recall)	86,8 %	Capta 92 de los 106 éxitos reales
Precisión comercial	58,6 %	92 de las 157 canciones promocionadas fueron éxito
Umbral de decisión	0,15	Se promociona si P(hit) ≥ 0,15
Pruebas automatizadas	9 de 9 aprobadas	Contrato de la API verificado con pytest

Nota. Valores tomados de la ficha del modelo y del informe del sprint. La línea base consiste en no promocionar ninguna canción (predecir siempre «no hit»).

2. Desarrollo técnico

Planteamiento y lógica económica. En la industria musical la inversión publicitaria se decide antes del lanzamiento de un sencillo, y los dos tipos de error tienen consecuencias económicas distintas:

- **Falso positivo (FP):** se promociona una canción que finalmente no es un éxito; se pierde la inversión publicitaria inicial (costo = 1).
- **Falso negativo (FN):** no se promociona una canción que tenía potencial de éxito; se pierde el ingreso por reproducciones, regalías y licencias (costo = 3).

El costo total se calcula como **Costo = 1 × FP + 3 × FN**. En el periodo de prueba hay 106 éxitos reales; una estrategia que nunca promociona los pierde todos, de modo que su costo es 106 × 3 = 318 unidades, valor que se toma como línea base. Según la ficha del modelo, los costos los fija quien responde por el presupuesto, no el modelo.

Tecnologías. El análisis y el modelo se desarrollan con Python 3.11.9, pandas 2.2.3, NumPy 2.2.6, scikit-learn 1.6.1 y joblib 1.6.0. El servicio utiliza FastAPI 0.115.12, Pydantic 2.11.4 y Uvicorn 0.34.2; las pruebas, pytest 8.3.5, httpx y requests. El panel interactivo emplea Streamlit y Plotly, y el despliegue se realiza en Render. Las versiones se fijan en requirements.txt porque el archivo modelo.pkl debe crearse con las mismas versiones con las que se ejecuta el servicio.

Datos y preparación. El conjunto de datos canciones.csv contiene 2000 registros y 13 columnas, sin valores faltantes. La variable objetivo es es_hit, que vale 1 cuando popularidad ≥ 70 (411 canciones, el 20,6 % del total; Figura 2). Para evitar fuga de información se excluyen dos variables: reproducciones_sem1, porque ocurre después del lanzamiento y no existe al momento de decidir, y popularidad, porque define es_hit y usarla como entrada equivaldría a conocer la respuesta. Las diez variables de entrada y los rangos que valida el servicio se resumen en la Tabla 2.

La partición es temporal: las canciones de 1995 a 2019 se usan para entrenamiento (1585 registros, 305 éxitos, 19,2 %) y las de 2020 a 2025 para prueba (415 registros, 106 éxitos, 25,5 %). Esta separación simula el escenario real de predecir canciones posteriores a las usadas para entrenar (Figura 3). Cabe notar que la proporción de éxitos es mayor en el periodo de prueba que en el de entrenamiento.

Tabla 2. Variables de entrada del modelo y rangos válidos en la API

Variable	Rango válido	Observación
bailabilidad	0 a 1	Característica de audio
energia	0 a 1	Característica de audio

Variable	Rango válido	Observación
valencia	0 a 1	Característica de audio
acustica	0 a 1	Característica de audio
tempo	40 a 250	Pulsos por minuto
duracion_min	0,5 a 15	Duración en minutos
volumen_db	−60 a 5	Volumen en decibelios
anio	1990 a 2035	Año de lanzamiento
colaboracion	0 a 1	Indica si hay colaboración (0 = no, 1 = sí)
genero	Texto	En los datos: pop, urbano, rock, electronica, indie

Nota. Los rangos corresponden al esquema Cancion del servicio; los campos adicionales están prohibidos y se rechazan con error 422 (por ejemplo, reproducciones_sem1, popularidad o es_hit).

Modelo. Se sustituyeron los modelos lineales de referencia por un algoritmo de ensamble no lineal basado en árboles de decisión, el HistGradientBoostingClassifier, que estima la probabilidad de que una canción sea un éxito. El entrenamiento respetó la estructura temporal de los datos mediante TimeSeriesSplit sobre el periodo 1995-2019. El pipeline entrenado, la ficha del modelo y las columnas de entrada se empaquetan en models/modelo.pkl con joblib (0,10 MB), y la ficha se imprime al finalizar el entrenamiento (Figura 8).

Regla de decisión y umbral. La regla del sistema es promocionar la canción cuando la probabilidad de hit estimada alcanza o supera el umbral de 0,15 (Figura 7). La documentación del sprint indica que este valor se identificó mediante validación cruzada temporal sobre los datos históricos y que, sobre la prueba 2020-2025, reduce el costo de 318 a 107 unidades (Figura 5). Como referencia, si las probabilidades estuvieran perfectamente calibradas, el umbral que minimiza el costo esperado con costos 1 y 3 sería 1 / (1 + 3) = 0,25. El umbral empleado es menor, lo que hace al sistema más propenso a promocionar: prioriza la cobertura de éxitos (recall de 86,8 %) a costa de la precisión (58,6 %), coherente con la mayor penalización del falso negativo.

Tabla 3. Desempeño del modelo frente a la línea base (prueba 2020-2025)

Métrica	Línea base	Modelo Sprint 2	Lectura comercial
Costo total	318	107	−66,3 % en pérdidas
Recall	0,0 %	86,8 %	92 de 106 éxitos captados
Precisión	No aplica	58,6 %	92 de 157 promociones acertadas
Canciones promocionadas	0	157 (37,8 %)	Del total de 415 de la prueba

Métrica	Línea base	Modelo Sprint 2	Lectura comercial
Falsos negativos (éxitos perdidos)	106	14	Costo 42 con el modelo
Falsos positivos (promociones fallidas)	0	65	Costo 65 con el modelo

Nota. Matriz de confusión del modelo: TN = 244, FP = 65, FN = 14, TP = 92 (suma 415). Costo del modelo = 65 × 1 + 14 × 3 = 107.

Servicio y despliegue. El servicio FastAPI detecta automáticamente la tarea del artefacto modelo.pkl y adapta sus esquemas de respuesta: en clasificación devuelve la probabilidad de éxito, la decisión de promoción y los metadatos de costos. Expone cuatro rutas (Tabla 4) y valida cada solicitud con Pydantic. El despliegue se define en render.yaml como un servicio web de Python (plan gratuito) con instalación mediante requirements.txt, arranque con Uvicorn y verificación de estado en /salud. Con el despliegue automático activado, cada cambio sincronizado en el repositorio remoto dispara la construcción del contenedor y la publicación de la API sin interrupción del servicio.

Tabla 4. Rutas expuestas por el servicio

Método	Ruta	Función
GET	/salud	Estado del servicio y del modelo (también usada como verificación de salud en Render)
GET	/modelo	Ficha técnica: qué predice, con qué datos, qué tan bien y qué decisión sostiene
POST	/predecir	Predicción y decisión para una canción
POST	/predecir_lote	Predicción y decisión para hasta 1000 canciones

Nota. Servicio publicado en https://demo-reg-clas-canciones-ctmb.onrender.com; la documentación interactiva (Swagger UI) está disponible en /docs. En el plan gratuito de Render el servicio puede suspenderse por inactividad y la primera solicitud posterior tardar más de lo habitual.

Panel interactivo. Sobre la API y el modelo se construyó un panel en Streamlit para que un usuario de negocio pruebe canciones sin usar código. El usuario captura artista, título, año, género, energía, valencia, acústica, bailabilidad, duración y volumen (el tempo y la colaboración se mantienen en valores por defecto: 120 BPM y sin colaboración). El panel carga modelo.pkl, calcula la probabilidad de hit y la compara con el umbral de la ficha: muestra una tarjeta verde con «Recomendado promocionar» o una naranja con «No promocionar», un radar con el perfil sonoro (energía, bailabilidad, valencia y acústica) y tres gráficas de posicionamiento: la distribución histórica de popularidad con su media (58,5 puntos), el número de canciones por año resaltando el año elegido y la relación entre bailabilidad y popularidad con su línea de tendencia.

Pruebas y calidad. Se mantuvieron y ejecutaron con éxito 9 de 9 pruebas automatizadas con pytest, que verifican el contrato de la API, los tiempos de respuesta (menos de 7 segundos para la suite completa) y la consistencia de los tipos de datos devueltos. Además, el script src.probar_servicio permite ejecutar una prueba de humo contra la URL pública del servicio.

Comparación entre sprints. La Tabla 5 resume cómo evoluciona el proyecto del Sprint 1 al Sprint 2. Ambos modelos se entrenan con canciones de 1995 a 2019 y se evalúan con datos de 2020 a 2025.

Tabla 5. Comparación entre el Sprint 1 y el Sprint 2

Característica	Sprint 1: regresión	Sprint 2: clasificación
Objetivo	Estimar la popularidad	Predecir si la canción será un hit
Variable objetivo	popularidad (0 a 100)	es_hit (1 si popularidad ≥ 70)
Modelo desplegado	HistGradientBoosting con características derivadas	HistGradientBoostingClassifier + umbral de costo mínimo
Variables excluidas	reproducciones_sem1, es_hit	reproducciones_sem1, popularidad
Métrica principal	MAE: 6,61 (línea base 10,44)	Costo: 107 (línea base 318)
Otras métricas	RMSE 8,35; R^2 0,616	Recall 86,8 %; precisión 58,6 %
Decisión	Promocionar si la popularidad esperada supera 65	Promocionar si P(hit) ≥ 0,15

Nota. Cifras del Sprint 1 tomadas del README y de la ficha publicada por el servicio en esa entrega; cifras del Sprint 2 tomadas de la ficha generada por src.entrenar_clasificacion.

Limitaciones y consideraciones. Los resultados deben leerse con las siguientes precauciones:

- **Datos sintéticos y asociaciones.** La ficha advierte que el modelo describe asociaciones en datos sintéticos; no constituye evidencia causal ni garantiza el mismo desempeño con canciones reales.
- **Dependencia de los costos.** El umbral de 0,15 es óptimo para costos de 1 (FP) y 3 (FN). Si el responsable del presupuesto modifica esos costos, el umbral debe recalcularse.
- **Una sola partición de prueba.** El desempeño se mide sobre 415 canciones de 2020 a 2025, un periodo con mayor proporción de éxitos (25,5 %) que el de entrenamiento (19,2 %). Con otra proporción de éxitos, la precisión y el costo variarían.
- **Referencia del taller.** El README del taller reporta como resultado de referencia un costo de 99 (regresión logística, umbral de 0,27) frente a la misma línea base de 318. El modelo del equipo obtuvo un costo de 107 con umbral de 0,15; ambos reducen de forma sustancial el costo de la línea base.
- **Entorno de ejecución.** El servicio debe ejecutarse con las versiones fijadas (scikit-learn 1.6.1) para cargar correctamente modelo.pkl; en el plan gratuito de Render la primera solicitud tras un periodo de inactividad puede tardar más.

3. Evidencias

A continuación se presentan las figuras que respaldan el análisis, en el orden en que se citan en el texto.

Demostración en Vivo en Swagger UI y Streamlit

```
C:\Users\xavie\Downloads\curso_Data_Analytics\demo-reg-clas-canciones>python -m
src.entrenar_clasificacion
Modelo guardado en
C:\Users\xavie\Downloads\curso_Data_Analytics\demo-reg-clas-canciones\models\modelo.pkl (0.10
MB)
{
  "nombre": "HistGradientBoosting + Umbral de Costos Mínimos",
  "tarea": "clasificacion",
  "objetivo": "es_hit: 1 si la canción alcanza 70 puntos de popularidad",
  "momento_prediccion": "antes del lanzamiento: la disquera decide cuánto invertir en promoción",
  "variables_entrada": [
    "bailabilidad",
    "energia",
    "valencia",
    "acustica",
    "tempo",
    "duracion_min",
    "volumen_db",
    "anio",
    "colaboracion",
    "genero"
  ],
  "variables_excluidas": {
    "reproducciones_sem1": "ocurre después del lanzamiento: no existe en el momento de decidir",
    "popularidad": "define es_hit: usarla como entrada equivale a conocer la respuesta"
  },
  "entrenamiento": {
    "periodo": "1995-2019",
    "n": 1585
  },
  "prueba": {
    "periodo": "2020-2025",
    "n": 415,
    "tipo": "temporal"
  },
  "linea_base": {
    "descripcion": "predecir siempre 'no hit'",
    "costo": 318
  },
  "desempeno": {
    "precision": 0.586,
    "recall": 0.868,
    "matriz_confusion": {
      "tn": 244,
      "fp": 65,
      "fn": 14,
      "tp": 92
```

```
    },
    "costo": 107
  },
  "decision": {
    "umbral": 0.15,
    "costos": {
      "falso_negativo": 3,
      "falso_positivo": 1
    },
    "regla": "promocionar si la probabilidad de hit supera el umbral",
    "nota": "los costos los fija quien responde por el presupuesto, no el modelo"
  },
  "limites": "describe asociaciones en datos sintéticos",
  "fecha": "2026-10-17",
  "versiones": {
    "python": "3.11.9",
    "scikit_learn": "1.6.1"
  }
}
```


Figura 1. Test session starts

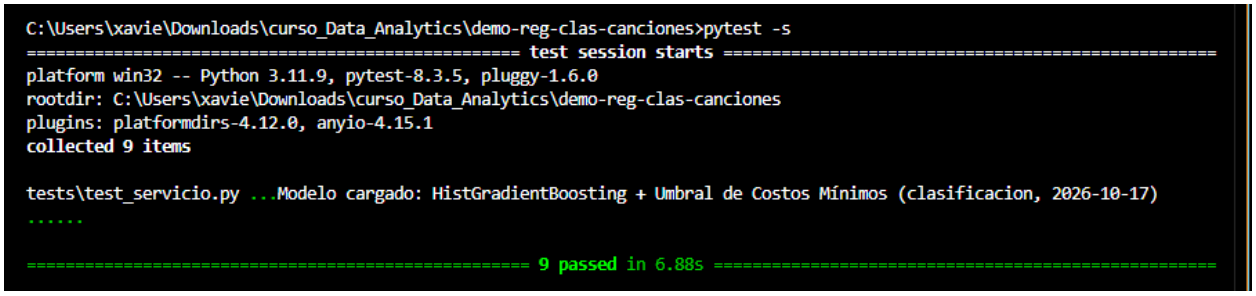
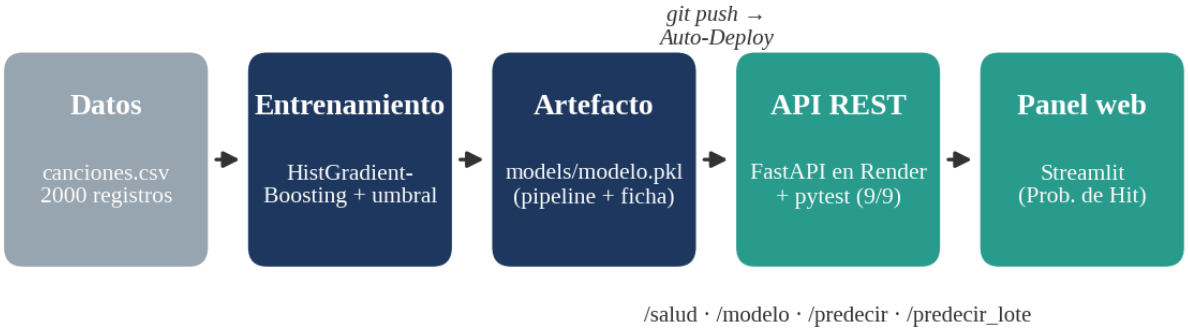


Figura 2. Panel de predicción y análisis musical

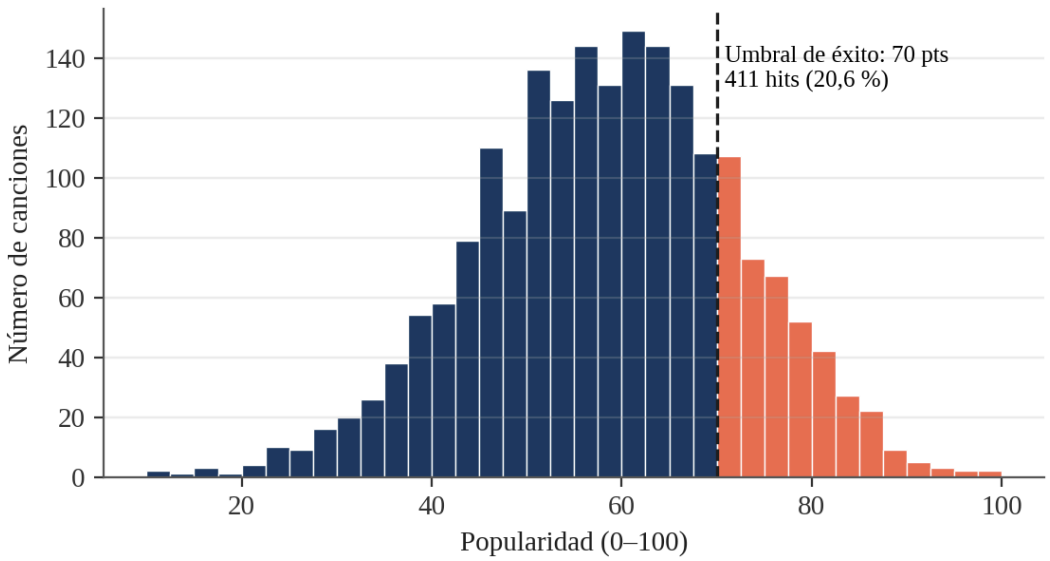


Figura 3. Arquitectura de la solución



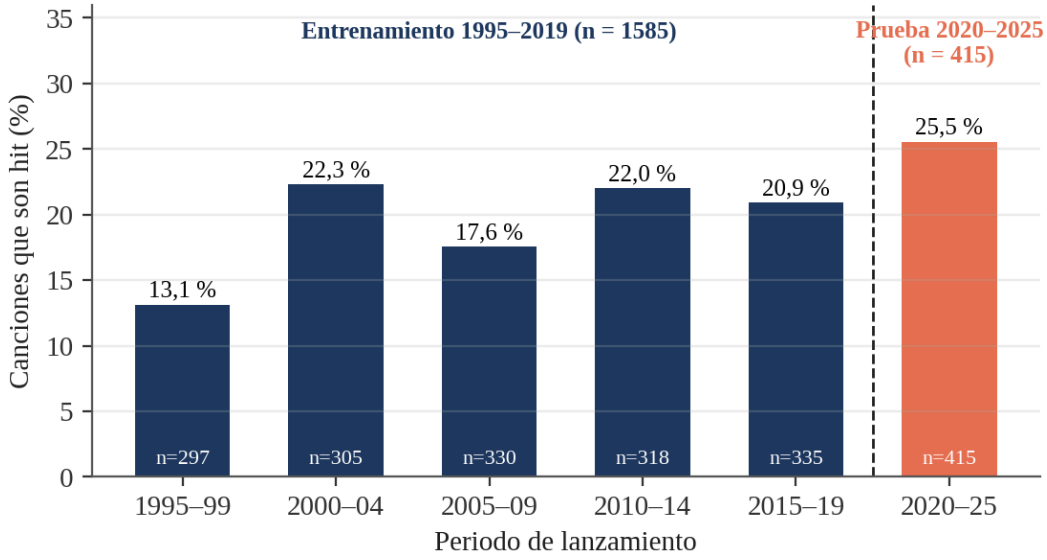
Nota. Flujo del sistema: el conjunto canciones.csv alimenta el entrenamiento del HistGradientBoostingClassifier; el resultado se empaqueta en models/modelo.pkl, que consume la API FastAPI desplegada en Render (con despliegue automático al sincronizar el repositorio y 9 pruebas automatizadas) y, a su vez, el panel de Streamlit. Elaboración propia a partir de la documentación del proyecto.

Figura 4. Distribución de la popularidad y definición de éxito (hit)



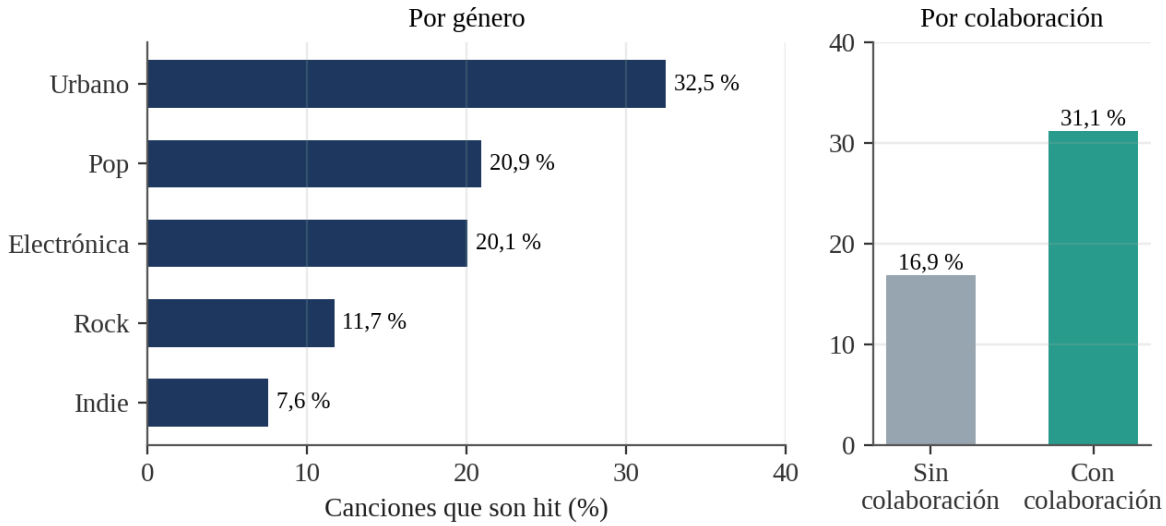
Nota. Histograma de la popularidad de las 2000 canciones del conjunto de datos. Las canciones con 70 puntos o más (411, el 20,6 %) se etiquetan como éxito (es_hit = 1). Elaboración propia a partir de canciones.csv.

Figura 5. Proporción de éxitos por periodo y partición temporal



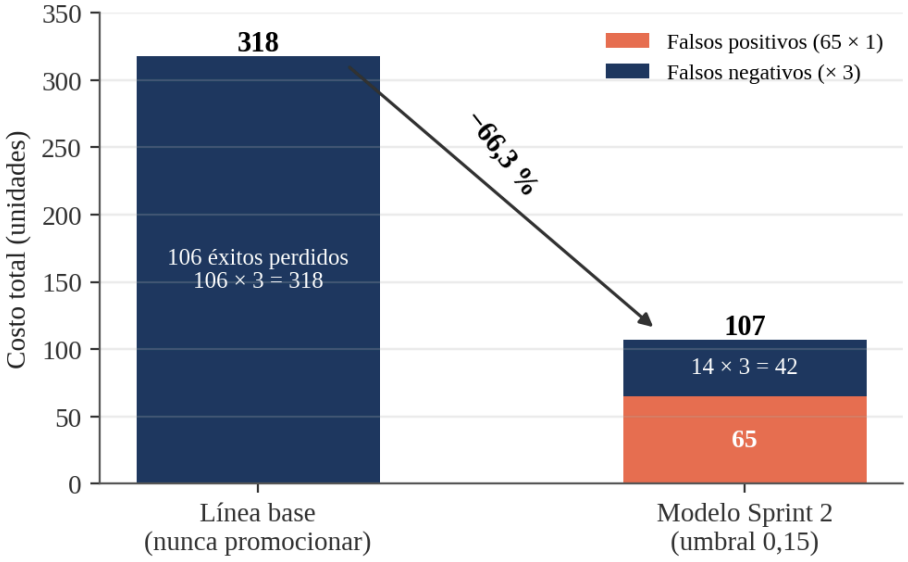
Nota. Porcentaje de canciones que son éxito por quinquenio de lanzamiento. El periodo 1995-2019 (n = 1585; 19,2 % de éxitos) se usa para entrenar y 2020-2025 (n = 415; 25,5 %) para evaluar. Elaboración propia a partir de canciones.csv.

Figura 6. Proporción de éxitos por género y por colaboración



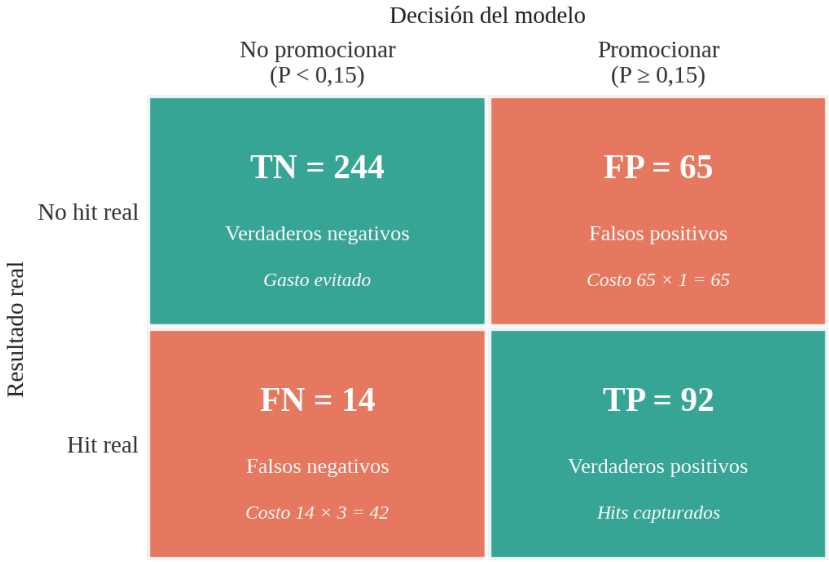
Nota. Tasa observada de éxito en todo el conjunto de datos: urbano 32,5 %, pop 20,9 %, electrónica 20,1 %, rock 11,7 % e indie 7,6 %; con colaboración 31,1 % frente a 16,9 % sin ella. Son comparaciones descriptivas sobre datos sintéticos y no implican causalidad. Elaboración propia a partir de canciones.csv.

Figura 7. Costo total: línea base frente al modelo del Sprint 2



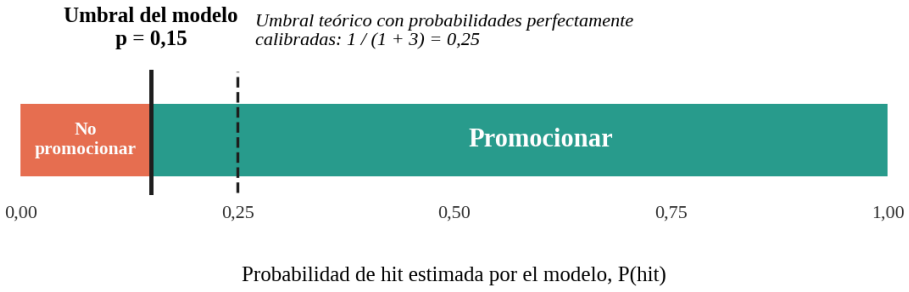
Nota. La línea base (nunca promocionar) pierde los 106 éxitos de la prueba, con un costo de 318 unidades. El modelo con umbral 0,15 incurre en 65 unidades por falsos positivos y 42 por falsos negativos (14 × 3), para un total de 107: una reducción del 66,3 %. Cifras de la ficha del modelo.

Figura 8. Matriz de confusión en la prueba temporal (2020-2025)



Nota. De 415 canciones: 244 verdaderos negativos, 65 falsos positivos, 14 falsos negativos y 92 verdaderos positivos. Recall = 92 / 106 = 86,8 %; precisión = 92 / 157 = 58,6 %. Cifras de la ficha del modelo.

Figura 9. Regla de decisión por umbral de probabilidad



Nota. El sistema recomienda promocionar cuando P(hit) ≥ 0,15. La línea punteada marca el umbral teórico de 0,25 (probabilidades calibradas, costos 1 y 3), solo como referencia.

Figura 10. Ficha del modelo generada al ejecutar src.entrenar_clasificacion

```
> python -m src.entrenar_clasificacion
Modelo guardado en models\modelo.pkl (0.10 MB)
{
  "nombre": "HistGradientBoosting + Umbral de Costos Mínimos",
  "tarea": "clasificacion",
  "objetivo": "es_hit: 1 si la canción alcanza 70 puntos de popularidad",
  "momento_prediccion": "antes del lanzamiento: la disquera decide cuánto invertir en promoción",
  "variables_entrada": ["bailabilidad", "energia", "valencia", "acustica", "tempo", "duracion_min",
"volumen_db", "anio", "colaboracion", "genero"],
  "variables_excluidas": {
    "reproducciones_sem1": "ocurre después del lanzamiento: no existe en el momento de decidir",
    "popularidad": "define es_hit: usarla como entrada equivale a conocer la respuesta"
  },
  "entrenamiento": { "periodo": "1995-2019", "n": 1585 },
  "prueba": { "periodo": "2020-2025", "n": 415, "tipo": "temporal" },
  "linea_base": { "descripcion": "predecir siempre 'no hit'", "costo": 318 },
  "desempeno": {
    "precision": 0.586,
    "recall": 0.868,
    "matriz_confusion": { "tn": 244, "fp": 65, "fn": 14, "tp": 92 },
    "costo": 107
  },
  "decision": {
    "umbral": 0.15,
    "costos": { "falso_negativo": 3, "falso_positivo": 1 },
    "regla": "promocionar si la probabilidad de hit supera el umbral",
    "nota": "los costos los fija quien responde por el presupuesto, no el modelo"
  },
  "limites": "describe asociaciones en datos sintéticos",
  "fecha": "2026-10-17",
  "versiones": { "python": "3.11.9", "scikit_learn": "1.6.1" }
}
```

Nota. Salida de consola del script de entrenamiento: tarea, variables, periodos, línea base, desempeño, regla de decisión con sus costos, límites y versiones (Python 3.11.9, scikit-learn 1.6.1).

4. Conclusión

El Sprint 2 convierte la estimación de popularidad en una decisión de negocio medible: promocionar o no una canción antes de su lanzamiento, teniendo en cuenta que perder un éxito cuesta el triple que una campaña fallida. Con un HistGradientBoostingClassifier, una partición temporal que evita la fuga de información y un umbral de 0,15, el modelo capta el 86,8 % de los éxitos de la prueba 2020-2025 y reduce el costo total de 318 a 107 unidades, es decir, un 66,3 %.

El ciclo completo queda verificado: preparación de los datos, entrenamiento y evaluación, empaquetado en un archivo .pkl, servicio FastAPI desplegado en Render con despliegue continuo, panel interactivo en Streamlit y 9 pruebas automatizadas aprobadas. Como límites, los resultados provienen de datos sintéticos, describen asociaciones y dependen de los costos fijados por quien responde por el presupuesto.

Anexo A. Guía de sustentación

Para la presentación final ante el docente se recomienda seguir esta estructura argumental en cuatro pasos.

Paso 1: Demostrar la alineación estratégica con el negocio

«En el Sprint 2 superamos la estimación de popularidad para enfocarnos en la decisión financiera de la disquera: ¿debemos o no promocionar una canción antes de lanzarla? Como perder un hit cuesta el triple que hacer una campaña no rentable, no buscamos maximizar la exactitud, sino minimizar el costo económico.»

Paso 2: Explicar la optimización del umbral (0,15)

«Utilizamos un HistGradientBoostingClassifier ajustado con TimeSeriesSplit sobre los años 1995-2019. Evaluando la función de costos sobre la prueba 2020-2025, con un umbral del 15 % capturamos el 86,8 % de los hits reales y bajamos el costo total de 318 a 107 unidades.»

Paso 3: Validar la arquitectura MLOps y el despliegue en la nube

«Todo el ciclo se empaquetó en un pipeline reproducible. La API en FastAPI adapta automáticamente la respuesta al detectar la tarea de clasificación. Contamos con 9 pruebas automatizadas en pytest en verde y la solución está desplegada en Render, con un panel en Streamlit.»

Paso 4: Demostración en vivo

Mostrar el endpoint GET /modelo en Swagger UI para verificar la ficha técnica en producción y abrir la aplicación de Streamlit para probar el comportamiento de la predicción al cambiar parámetros sonoros.

Anexo B. Preguntas probables en la sustentación

¿Por qué no se usa la exactitud (accuracy) como métrica?

Porque con 106 éxitos entre 415 canciones, una línea base que nunca promociona alcanzaría cerca de 74,5 % de exactitud (309 de 415) con un recall de 0 %, sin aportar valor alguno. El modelo alcanza 81,0 % de exactitud (336 de 415), pero lo relevante es el costo: 107 frente a 318.

¿Cómo se evitó la fuga de información?

Se excluyeron reproducciones_sem1 (posterior al lanzamiento) y popularidad (define el objetivo), se partió el tiempo en entrenamiento 1995-2019 y prueba 2020-2025, y se usó TimeSeriesSplit para la validación.

¿Por qué el umbral es 0,15 y no 0,5?

Porque los errores no cuestan igual. Un umbral de 0,5 es adecuado cuando ambos errores pesan lo mismo; aquí el falso negativo cuesta 3 y el falso positivo 1. Con probabilidades calibradas el umbral teórico sería 0,25, y el valor 0,15 se obtuvo empíricamente al minimizar el costo, a costa de una menor precisión (58,6 %).

¿Qué ocurre si cambian los costos de la disquera?

El umbral debe recalcularse. Como indica la ficha del modelo, los costos los fija quien responde por el presupuesto, no el modelo.

¿Se puede usar el modelo para decisiones reales?

Con cautela. Los datos son sintéticos y el modelo describe asociaciones, no relaciones causales. Para un uso real habría que reentrenarlo con datos reales y validar el umbral con ellos.